

Python:-

```
print("shani")
```

```
name = "shani"
```

```
age = 23
```

```
is-adult = False
```

input:-

~~input~~

```
name = input("what is your name?")
```

```
print("Hello" + name)
```

```
old-age = int(input("enter your old age:"))
```

```
new-age = old-age + 2
```

```
print(new-age)
```

concatination & in string is possible.. but not int?

Floats

```
number = float(18)
```

```
print(number)
```

output = 18.0

```
number1 = 23
```

```
number2 = 26
```

```
add = int (number1 + number2)
```

```
print(add)
```

name = "roshan tasawar"

print(name.upper()) // (name.lower())

output: ROHAAN TASAWAR.

Find:-

name = "R⁰o¹h²a³a⁴n T⁶a⁷s⁸a⁹w¹⁰a¹¹r¹²"

print(name.find('a')) = output = 4 index

name = "Tony Stark"

print(name.replace("Tony Stark", "Iron Man"))

keywords:-

print ✓

Arithmetic operators:- $\ast, +, -, \%$

shortcuts:-

i = 5

i = i + 2

i += 2

operator

precedence

(kis ka
zayada priority
hoga)

result = (2 + 3 * 5)

then 2

Comments

here is a comment.

Comparison operators:-

print (3 > 2)

if-else:-

```
age = 19 } number = int(input("number is"))  
if age >= 18:
```

```
    if number > 0:
```

```
        print("The number is positive.")
```

```
    elif number < 0:
```

```
        print("The number is negative.")
```

```
    else:
```

```
        print("The number is zero.")
```

Calculator in Python:-

~~def (n, y)~~

```
def add(n, y):
```

```
    return n + y
```

```
def subtract(n, y):
```

```
    return n - y
```

```
def multiply(n, y):
```

```
    return n * y
```

```
def divide(n, y):
```

```
    if y == 0:
```

```
        return "Error! Division by zero"
```

```
    return n / y
```

```
print("select operations")
```

```
print("1. Add")
```

```
print("2. Subtract")
```

```
print("3. Multiply")
```

```
print("4. Divide")
```

```
while True
```

```
    choice = input("Enter choice 1/2/3/4")
```

```
    if choice in ('1', '2', '3', '4'):
```

```
        num1 = float(input("Enter a number:"))
```

```
        num2 = float(input("Enter a number:"))
```

```
        if choice == '1':
```

```
            print(f"{num1} + {num2} = {add(num1, num2)}")
```

```
        elif choice == '2':
```

```
            print(f"{num1} - {num2} = {subtract(num1, num2)}")
```

```
        elif choice == '3':
```

```
            print(f"{num1} * {num2} = {multiply(num1, num2)}")
```

```
        elif choice == '4':
```

```
            next_calculation = input("Do you want to perform  
another calculation? (yes/no): ")
```

```
            if next_calculation.lower() != 'yes':
```

```
                break
```

```
            else:
                print("Invalid input")
```


Loops:- while

i = 1

while i <= 5:

print(i)

i = i + 1 or i = i + 1

i = 1

while i <= 5:

print(i * "x")

i = i + 1

decrement (i - 1)

list:-

Marks = [95, 98, 97]

print(marks[-1]) output // 97

OR print(marks[1:3]) output // 98, 97

for score in marks:

print(score)

output = 95, 98, 97

add new marks:-

marks = [95, 98, 97]

marks.append(99, 98)

print(marks):

output = [95, 98, 97, 99, 98]

starting

marks.insert(0, 99)

functions

lambda functions:- (koie name deni ki zaroorat nahi hai)

are small, anonymous functions
define using the lambda keyword.
 $\text{lambda } x: x**2$

Example- Filter:-

numbers = [1, 20, 31, 40, 51, 60, 71, 80, 91, 100]

def even(x):

return $x \% 2 == 0$

evens = list(filter(even, numbers))

print("Even numbers:", evens).

output :- 20, 40, 60, 80, 100

(filter, sorted, map)
mostly used
lambda fun

Ex 2 Map:-

numbers : [1, 2, 3, 4, 5]

squared-numbers = list(map(lambda $x: x**2$, numbers))

print(squared-numbers)

output :- 1, 4, 9, 16, 25

filter
↑ $x: x \% 2 == 0$
{ searching by logic }
use list

Sorted:-

people = [("Alice", 30), ("Bob", 25), ("Charlie", 35)]

sorted_people = sorted(people, key = lambda $x: x[1]$)

print(sorted_people)

x: 7[9]

another example of sorted:-

students = [

{ "name": "Alice", "grade": 85 },

{ "name": "Bob", "grade": 92 },

{ "name": "Jani", "grade": 77 }]

sorted_students = sorted(students, key=lambda,

x: x["grade"])

print(sorted_students)

CS

Excel file main searching & sorting karna
Muskil hota ha

CSV Files:-

stands for Comma separated ^{values} files.
are commonly used to store tabular (in table form rows & columns) data.

python csv module makes it easy to read & write csv files.

A csv file is saved with a .csv extension.
csv file is used for Data Analysis, spreadsheet Applications and E-commerce & inventory Management.

open csv file in python.

A csv file can be opened using the built-in function open() with the appropriate mode like 'r' for reading, 'w' for writing, or 'a' for appending.

It's better to use the with statement that automatically handles the closing of the file even if any error occurs.

Import CSV
En: import csv

with open('example.csv', mode='r') as file:

csv-reader = csv.reader(file)

next(csv-reader, None)

for row in csv-reader:

print(row)

Writing CSV

import csv

```
data = ["Name", "Age", "Department"],  
       ["Alice", "30", "HR"],  
       ["Bob", "24", "IT"]
```

with open('output.csv', mode='w', newline='')
as file:

```
csv_writer = csv.writer(file)
```

```
csv_writer.writerow(data)
```

Numpy :-

stands for numerical python. It is a powerful tool in python that helps you work with numbers more easily. It support for large Multi-dimensional arrays and matrices.

Installation:

pip install numpy

import numpy

Array:-

Array is like a container that holds a collection of items. Array are useful for organizing & managing data in structured way.

Python

Numpy Array

↳ Not Directly store data.
store one time string data or
number data.

In Numpy you should first
install & import then he
should work

Numpy Array are optimized
for numerical computations &
are much faster than python
list, especially for large
Data sets.

all elements in numpy
array must be of the same
data type

Lists

↳ Directly data store in list

↳ Not import Module/packages

↳ List is in-built datatype.

↳ List capable for numerical
operations but less efficiency

Numpy array can be Multi-
dimensional, which is useful for
matrix operations & handling
complex data structures

Numpy

```
import Numpy as np
x = np.array [1, 2, 3, 4]
print(x)
```

~~type~~ (print(type(x)))

output = ~~[1, 2, 3, 4]~~ [1, 2, 3, 4]

~~Array~~ list

y = [1, 2, 3, 4]

print(y)

print(type(x))

output:-

[1, 2, 3, 4]

l = []

for i in range(1, 5):

int = ^{int} input("Enter: ")

l.append(int-1)

print(np.array(l))

output = ['12', '13', '14', '15']

~~12 13 14 15~~ [12 13 14 15]

Dimension in
Arrays.

→ 1D, → 2D

→ 3D

→ Higher

Dimension
Arrays

1D:- [1 2 3 4]

2D:- [[1 2 3 4]]

3D:- [[[1 2 3 4]]]

{ ndim
use to find
calculate
dimension }

22

arr2 = np.array([[1,2,3,4],[1,2,3,4]])
 print(arr2)
 print(arr2.ndim)

3D

arr3 = np.array([[[1,2,3,4],[1,2,3,4],[1,2,3,4]]])
 print(arr3)

Higher Dimension

arrn = np.array([1,2,3,4], ndmin = 10)
 print(arrn)

output

[[[[[[[[[[[2 3 4]]]]]]]]]]]]]

Create Numpy Array using Numpy Function

Array filled with 0's

import numpy as np
 ar_zero = np.zeros(4)
 print(ar_zero)

output [0, 0, 0, 0]

ar_zero = np.zeros(4)

ar_zero1 = np.zeros((3,4))

print(ar_zero)

print()

print(ar_zero1)

output

[0 0 0 0]

[[0 0 0 0]

[0 0 0]

[0 0 0]]

Range:-

```
ar-rn = np.arange(4)
```

```
print(ar-rn)
```

output [0 1 2 3]

Diagonal

```
ar-dia = np.eye(3) // k
```

```
print(ar-dia)
```

output $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

linSPACE:-

```
ar-lin = np.linspace(0, 20, num=5)
```

```
print(ar-lin)
```

output [0. 5. 10. 15. 20.]

Create numpy Arrays with Random No

rand():- The function is used to generate a random value between 0 to 1.

randn():-

The function is used to generate a random value close to zero

This may return +ve or -ve numbers as well.

Ex:- import numpy as np

```
var = np.random.rand(4)
```

```
print(var)
```

output [0.4577441 0.3614105 0.710625
0.1334147]

```
var 2 = np. random. randn(5)
```

```
print (var 2)
```

output $[0.274 \quad 0.246 \quad -1.343 \quad -0.265 \quad -0.25]$

rand() - the function for doing random sampling in numpy. close to 0 & less than 1.

```
var 3 = np. random. rand(4)
```

```
print (var 3)
```

$[0.025 \quad 0.006 \quad 0.917 \quad 0.530]$

randint() -

the function is used to generate a random number within a specified range.

```
import random
```

```
random_number = random. randint (1, 10)
```

```
print (random_number)
```

output 7

Data Types

```
Var = np.array([1.0, 1.2, 1.3])
```

```
print("Data Type :", var.dtype)
```

output: float64.

Arithmetic operations in Numpy Arrays.

$a+b$:- `np.add(a,b)`

$a \times b$:- `np.power(a,b)`

$a-b$:- `np.subtract(a,b)`

```
import numpy as np
```

```
var = np.array([1, 2, 3, 4])
```

```
varadd = var + 3 // np.add(4,2)
```

```
print(varadd)
```

```
var1 = np.array([1, 2, 3, 4])
```

```
var2 = np.array([5, 6, 7, 8])
```

```
varadd = var1 + var2
```

```
print(varadd)
```

output [4 5 6 7]

output [6 8 10 12]

2D Arrays

```
var21 = np.array([[1, 2, 3, 4], [1, 2, 3, 4]])
```

```
var22 = np.array([[5, 6, 7, 8], [5, 6, 7, 8]])
```

```
varadd = np.add(var21, var22)
```

$= var21 + var22$

```
print(var21)
```

```
print()
```

```
print(var22)
```

```
print()
```

```
varadd = var21 + var22
```

```
print(varadd)
```

Arithmetic Functions:-

`np.min(x)` `np.max(x)` `np.argmax(x)`
`np.sqrt(x)` `sin(x)` `cos(x)`
`np.cumsum(x)`

```

Ex) var = np.array([45, 67, 89, 00, 45, 23, 99])
varmax = np.max(var)
print(varmax)      output = 99
    
```

Shape:- (Var. shape)

```

var = np.array([1, 2, 3, 4], [1, 2, 3, 4])
print(var)
print( )      output = (2, 4)      row ↑ column ↑
print(var.shape)
    
```

numpy main kabhi bi blank row nahi banti hai

Reshape

```
var2 = np.array([1, 2, 3, 4, 5, 6])
```

```
x = var2.reshape(3, 2)
```

```
print(x)
```

```

output [[1, 2]
        [3, 4]
        [5, 6]]
    
```

Dimension ^{size} should be same otherwise broadcasting occur

Indexing & slicing.

Indexing.

Indexing in Numpy allows you to access individual elements of an array using their position (index). also use in (-1) indexing.

Ex

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr[0]) // output: 1
```

```
print(arr[4]) // output: 5
```

slicing

slicing is a way to access a range of elements in an array. It uses the colon (:) operator to specify the start, stop & step values.
Syntax: `array[start:stop:step]`

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr[1:4]) // output: [2, 3, 4]
```

```
print(arr[:3]) // [1, 2, 3]
```

```
print(arr[::2]) // [1, 3, 5]
```

Concatenates:-

import numpy as np

var1 = np.array([1, 2, 3, 4])

var2 = np.array([9, 8, 7, 6])

arr = np.concatenate((var1, var2))

print(arr)

output [1 2 3 4 9 8 7 6]

stack

var1 = np.array([1, 2, 3, 4])

var2 = np.array([9, 8, 7, 6])

a-new = np.stack((var1, var2), axis=1)

print(a-new)

output
 [[1 9]
 [2 8] [3 7] [4 6]]

split:-

var = np.array([1, 2, 3, 4, 5, 6])

print(var)

arr = np.array_split(var, 3)

print()

print(arr)

print(arr[0])

output

[1 2 3 4 5 6]

[array([1, 2]), array([3, 4]),
 array([5, 6])]
 1 1 1

Data Manipulating & cleaning

Data Analysis:-

Pandas:- is the process of examining, cleaning, transforming & interpreting data to discover useful information, draw conclusions and support decision-making.

Pandas:-

is a powerful library for data manipulation & analysis. It provides data structures such as series (1D) & Data frames. import pandas as pd

→ a = pd.Series()
print(a)

Series:-

Think of a series as a single column of data. It's like a list but with labels (index) for each element. ~~end~~

Dataframes:-

is like a table with rows & columns. Each column is a series, & the Data Frame holds multiple series together.

panel (Diagram by layer)

other way pass other things
such as dictionary

Date 1 / 20

Series - ^{list}

x = [3, 4, 5, 6, 7, 8]

var = pd.Series(x)

print(var)

print(var[3]) // = output 6

0	3
1	4
2	5
3	6
4	7
5	8

// change index

var = pd.Series(x, index = ['a', 'b', 'c'], dtype = float)

print(var)

a 3.0
b 4.0
c 5.0
output

dic = { "name": ['python', 'c', 'c++', 'java'], "rank": [1, 4, 3, 2], "pos": [12, 13, 14, 15] }

output name [python, c, c++, java]

rank [1, 4, 3, 2]

var = pd.Series(dic)

print(var)

dtype: object (because max data)

In pandas

Directly work kr statky han without using broadcasting

s1 = pd.Series([12, index = [1, 2, 3, 4, 5, 6, 7]])

s2 = pd.Series([12, index = [1, 2, 3, 4]])

print(s1 + s2)

output
1 24.0
2 24.0
3 24.0
4 24.0
5 NaN
6 NaN
7 NaN

note number

In pandas not error occur
but in numpy error occurs

lock:- label based

index based search

Mon Tue Wed Thu Fri Sat

list/dict

Date: / / 20

Data Frames:-

imp $l = [1, 2, 3, 4, 5]$

var = pd.DataFrame(l)

~~print(var)~~
print(var)

output

0	1
1	2
2	3
3	4
4	5
5	6

$d = \{ "a": [1, 2, 3, 4], "s": [1, 2, 4, 6] \}$

// should same elements otherwise you face the error.

var = pd.DataFrame(d)

print(var) // print(type(var))

output

	a	s
0	1	1
1	2	2
2	3	3
3	4	6
4		

When you see in column in 2 or 3 index/dict then

var = pd.DataFrame(d, columns=["a"])

print(var)

When you change the index then

var = pd.DataFrame(d, columns=["a"], index=["a", "s", "d", "f"])

output

	a
a	1
s	2
d	3
f	4

When you get particular data then

print(var["a"][3])

then output = 4

Arithmetic operations:-

```
var = pd.DataFrame({"A": [1, 2, 3, 4], "B": [5, 6, 7, 8]})
var["C"] = var["A"] + var["B"]
print(var["C"])
```

overall same

2nd

```
var1 = pd.DataFrame({"A": [10, 20, 30, 40],
                      "B": [15, 16, 17, 18]})
```

```
var1["python"] = var1["A"] < 20
```

```
var1["python-1"] = var1["B"] >= 16
```

output

	A	B
0	10	15
1	20	16
2	30	17
3	40	18

var1	A	B	python	python-1
0	10	15	True	False
1	20	16	True	True
2	30	17	False	True
3	40	18	False	True

Delete & Insert

```
var = pd.DataFrame({"A": [1, 2, 3, 4, 5], "B": [9, 8, 7, 6, 5]})
```

var new table name

Then

```
var.insert(1, "python", var["A"])
```

var

	A	python	B
0	1		9
1	2		8
2	3		7
3	4		6
4	5		5

No. of
Data
should be
equal

limited data copy (slicing)

```
var["python-1"] = var["A"][:3]
```

var

Delete:- ^{Delete}
[↑]
 var.pop("B")
 var

var = var.pop("B")
 kon se data ap ny
 delete krna ha?

CSV File in Pandas:-

data = {"A": [1, 2, 3, 4, 5, 6], "B": [1, 2, 3, 4, 5, 6], "C": [1, 2, 3, 4, 5, 6]}

Write

d = pd.DataFrame(data)

print(d)

d.to_csv("Test-new.csv")

open without index

d.to_csv("Test-new1.csv", index=False)

change header

d.to_csv("Test-new1.csv", index=False,

header = [1, 2, 3])

Merge:- Both DataFrames one part is common

var1 = pd.DataFrame({"A": [1, 2, 3, 4], "B": [11, 12, 13, 14]})

var2 = pd.DataFrame({"A": [1, 2, 3, 4], "C": [21, 22, 23, 24]})

pd.merge(var1, var2, on="A")

output

	A	B	C
0	1	11	21
1	2	12	22
2	3	13	23
3	4	14	24

pd.merge(var2, var1, on="A")

A = [1, 2, 3, 4] & var2 = [1, 2, 3, 5]

Then ~~when~~ output [1, 2, 3] only show

same/common data.

how = "outer"
"inner"
"left"

(indicator = True)

⊙ use check which parameter merge or not,

⊙ see All data in one time.

pd.merge(var1, var2, left_index=True, right_index=True)

⊙ change name, id

pd.merge(var1, var2, left_index=True, right_index=True, suffixes=("_name", "_id"))

output

A_name	B_name	A_id	B_id
--------	--------	------	------

Concatination

Mon Tue Wed Thu Fri Sat

Date: ___/___/20

Sr1 = pd.Series([1, 2, 3, 4])

same as
DataFrame.

Sr2 = pd.Series([11, 22, 33, 44])

pd.concat([Sr1, Sr2], axis=1, keys=["d1", "d2"])

output

Group by:-

```
import pandas as pd dic
var = pd.DataFrame({"name": ["a", "b", "c", "d",
"a", "b", "a", "b", "a", "c", "c", "d"], "s-1":
[12, 13, 14, 12, 13, 14, 15, 25, 16, 10, 9, 34],
"s-2": [23, 24, 25, 26, 27, 28, 29, 30, 25, 34, 35, 36]})
```

```
Var
var-New = var.groupby("name")
Var-New
```

loop

```
for x, y in var-New:
    print(x)
    print(y)
    print()
```

single Data (a)

```
var-New.get_group("a")
```

Min, Max

var-New.min()

var-New.max()

Mean

var-New.mean()

Convert into list

li = list(var-New)

li (print)

output a			
	name	s-1	s-2
0	a	12	23
4	a	13	24
6	a	15	29
8	a	25	25
b			
	name	s-1	s-2
1	b	13	24
5	b	24	28
7	b	23	26
c			
	name	s-1	s-2
2	c	14	25
9	c	16	34
10	c	10	35
d			
	name	s-1	s-2
3	d	12	26
11	d	34	56

output d			
	name	s-1	s-2
3	d	12	26
11	d	34	56

Join & Append

Import pandas as pd

var1 = pd.DataFrame({"A": [1, 2, 3, 4], "B": [11, 12, 13, 14]})

var2 = pd.DataFrame({"C": [10, 20, 30, 40], "D": [11, 22, 33, 44]})

var1 = join(var2)

output
index

	A	B	C	D
0	1	11	10	11
1	2	12	20	22
2	3	13	30	33
3	4	14	40	44

Agar hum var-2 sy data

data kam kr den

to us jga par nan a. jay

jo output

change index

var1 = pd.DataFrame({"A": [1, 2, 3, 4], "B": [11, 12, 13, 14]})

index: ["a", "b", "c", "d"]

Agar ap sirf var1 Main index den gy to

var2 main nan show ga is lye ap ko

var2 Main be index dikhna / declare krna hoga.

var2 = pd.DataFrame({"C": [10, 20, 30, 40], "D": [11, 22, 33, 44]})

index: ["a", "b", "c", "d"]

output

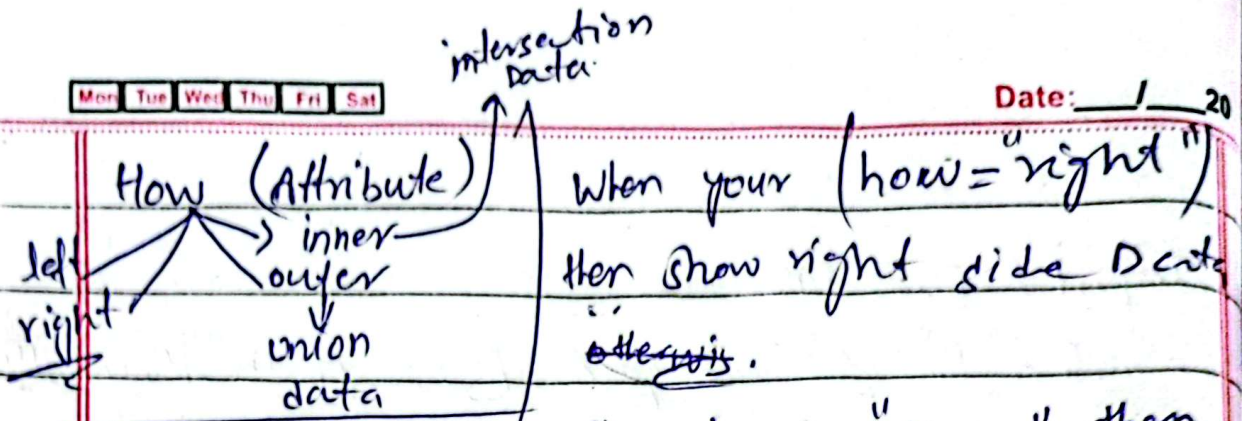
var1.join(var2)

split data then

var2.join(var1)

	A	B	C	D
a	1	11	10	11
b	2	12	20	22
c	3	13	30	33
d	4	14	40	44

Agar var2 Main du values hain to var1 ki bi du hi value show ho gey.



When you write how = "inner" then show common data.

When you see ^{data} same column but data set different Mean B is [1, 2, 3, 4] & and B [1, 2] then use suffix.

var 2: join(var1, how = "outer", suffix = "-")

then show all Data. ^{left} otherwise you use ^{right}.

output

	c	B-12	A	B
a	10.0	11.0	1	11
b	20.0	22.0	2	12
c	NaN	NaN	3	13
d	NaN	NaN	4	14

Append:-

Join ky andar ap ko error
face krna parta tha jab do 2 column
naam same hoon Tab, Tab ap ko
suffix use krna parta tha
but Append Main aisi nahi hai.

Same date join

`var1.append(var2)` then jahan par
date ha wahan data other jahan nahi hai
wahan NaN.

Agar ap ko index true it mean ek
pattern main show karana ha tab

`var1.append(var2, ignore_index = True)`

Reshape File use function.
pivl & melt.

Melt:-

import pandas as pd

var = pd.DataFrame({"days": [1, 2, 3, 4, 5, 6], "eng": [10, 12, 14, 15, 16, 12], "maths": [17, 18, 19, 13, 14, 16]})

var	index	variable	value
var	0	days	1
	1	days	2
	2	days	3
	3	days	4
	4	days	5
	5	days	6

ID

Front indexing (ID) by type.

pd.melt(var, id_vars=["eng"])

then output is id- vars

index	eng	variable	value
0	10	days	1
1	12	days	2
2	14	days	3
3	16	days	4
4	12	days	5
5	12	days	6
6	17	maths	17
7	18	maths	18
8	19	maths	19
9	13	maths	13
10	14	maths	14
11	16	maths	16

change var - name

pd.melt(var, id_vars=["eng"], var_name="pythen", value_name="shani")

Pivot

var: pd.DataFrame ({ "days": [1, 2, 3, 4, 5, 6],
 'stud-1': ["a", "b", "c", "b", "b", "c"],
 "eng": [10, 12, 14, 15, 16, 12], "maths": [17, 18, 19, 13, 14, 16] }

var

output

	days	st-name	eng	maths
0	1	a	10	17
1	2	b	12	18
2	3	c	14	19
3	4	a	15	13
4	5	b	16	14
5	6	c	12	16

Minimum two arguments must given.

var. pivot(index = "days", column = "stud-1")

output

eng Fill your ↑↑
 st-name a b c Date a b c
 days

1	10.0	NaN	NaN	17.0	NaN	NaN
2	NaN	12.0	NaN	NaN	18.0	NaN
3	NaN	NaN	14.0	NaN	NaN	19.0
4	15.0	NaN	NaN	13.0	NaN	NaN
5	NaN	16.0	NaN	NaN	14.0	NaN
6	NaN	NaN	12.0	NaN	NaN	16.0

particular
get Data (column)

Mon Tue Wed Thu Fri Sat

Date: ___/___/20___

When you get only one value

var. pivot (index = "days", columns = "st-name",
value = "eng", aggfunc = "mean",
margins = "true")

aggfunc() (Mean, sum, count)

margin = margins = true.

Types of learning:-

supervised learning:-

data with labels. (Answer is provided)

unsupervised learning

data without labels.

Reinforcement learning:-

states & actions.

Machine learning & Descriptive statistics

Descriptive statistics

Summarizes & describes features of a Dataset. Measures such as mean, median, mode, standard deviation are commonly used.

Machine learning:-

uses algorithms to learn from data & make predictions or decisions.

Includes supervised, unsupervised & reinforcement learning

Scikit-learn Toolkit

is a popular machine learning library in python. It provides tools for classification, regression, clustering & dimensionality reduction.

Data Dimensionality:-

refers to the number of features (or variables) in a dataset. In simple words, it's like the number of columns in a table.

Low-dimensional Data:-

Think of a table with just a few columns (features) like age, height & weight. It's easy to visualize & analyze.

High-Dimensional Data:-

Imagine a table with hundreds or thousands of columns (features). This can make analysis more challenging & computationally intensive.

Example:-

In an image dataset, each pixel can be considered a feature. An image with 1000×1000 pixels has 1,000,000 dimensions.

Reducing Dimensionality:-

PCA (principal component Analysis) helps reduce the number of dimensions while maintaining important information.

Pipelines:-

Automates workflows, combining preprocessing & model training into a single object.

Scikit ~~learn~~ learn:-

from sklearn import datasets.

have many types & also given
→ built-in

• fit() train models,

• predict() making predictions.

train_test_split():-

that helps you divide your dataset into two parts: one for training & one for testing it

Accuracy()

measure of how well your model's predictions match actual outcomes.

syntax: from sklearn.metrics import accuracy_score.

Y-true = [0, 1, 1, 0, 1] Actual &

Y- pred = [0, 1, 0, 0, 1] prediction labels

accuracy = accuracy_score(y-true, y-pred)

print('Accuracy: {accuracy}')

output = 0.8 Accuracy